

WEEK-5

TASK-1:

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <limits.h>

#include <sys/stat.h>

#include <errno.h>


#define INPUT_SIZE 256


/* Display current working directory */
void show_current_directory(void)
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("getcwd");
        return;
    }
}
```

```

    printf("Current directory: %s\n", cwd);
}

/* Check whether the path exists and is a directory */
int validate_directory(const char *path)
{
    struct stat info;

    if (stat(path, &info) != 0)
    {
        return 0;
    }

    if (!S_ISDIR(info.st_mode))
    {
        return 0;
    }

    return 1;
}

/* Change directory */
int change_directory(const char *path, char *previous_dir)
{
    char current_dir[PATH_MAX];

```

```
/* Get current directory before changing */
if (getcwd(current_dir, sizeof(current_dir)) == NULL)
{
    perror("getcwd");
    return 1;
}

/* Validate target path */
if (!validate_directory(path))
{
    printf("Error: '%s' is not a valid directory.\n", path);
    return 1;
}

/* Change directory */
if (chdir(path) != 0)
{
    perror("chdir");
    return 1;
}

/* Save previous directory */
strcpy(previous_dir, current_dir);

printf("Directory changed successfully.\n");
printf("Previous directory: %s\n", previous_dir);
```

```

    show_current_directory();

    return 0;
}

/* Go to previous directory */
int go_previous_directory(char *previous_dir)
{
    char current_dir[PATH_MAX];
    char temp[PATH_MAX];

    if (previous_dir[0] == '\0')
    {
        printf("No previous directory available.\n");
        return 1;
    }

    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(temp, current_dir);

```

```

if (chdir(previous_dir) != 0)
{
    perror("chdir");
    return 1;
}

strcpy(previous_dir, temp);

printf("Moved to previous directory.\n");
show_current_directory();

return 0;
}

int main(void)
{
    char input[INPUT_SIZE];
    char previous_dir[PATH_MAX] = "";

    printf("==== DIRECTORY NAVIGATION =====\n");
    printf("Commands:\n");
    printf(" pwd      - Show current directory\n");
    printf(" cd PATH   - Change directory\n");
    printf(" cd -      - Go to previous directory\n");
    printf(" exit      - Exit program\n");

```

```
while (1)
{
    printf("\nav-shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        continue;
    }

    /* exit */
    if (strcmp(input, "exit") == 0)
    {
        printf("Exiting navigation shell...\n");
        break;
    }

    /* pwd */
    if (strcmp(input, "pwd") == 0)
```

```

{
    show_current_directory();
    continue;
}

/* cd - */
if (strcmp(input, "cd -") == 0)
{
    go_previous_directory(previous_dir);
    continue;
}
if (strncmp(input, "cd ", 3) == 0)
{
    char *path = input + 3;

    if (strlen(path) == 0)
    {
        printf("Error: directory path is missing.\n");
        continue;
    }
    change_directory(path, previous_dir);
    continue;
}
printf("Invalid command: %s\n", input);
}
return 0;

```

}

```
root@LAPTOP-RJFD8EBT:~# nano cd_navigation.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g cd_navigation.c -o cd_navigation
root@LAPTOP-RJFD8EBT:~# ./cd_navigation
===== DIRECTORY NAVIGATION =====
Commands:
  pwd          - Show current directory
  cd PATH      - Change directory
  cd -         - Go to previous directory
  exit         - Exit program

nav-shell> pwd
Current directory: /root

nav-shell> cd /tmp
Directory changed successfully.
Previous directory: /root
Current directory: /tmp

nav-shell> pwd
Current directory: /tmp

nav-shell> cd -
Moved to previous directory.
Current directory: /root

nav-shell> exit
Exiting navigation shell...
root@LAPTOP-RJFD8EBT:~# |
```

TASK-2:

TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g builtin_shell.c -o builtin_shell
root@LAPTOP-RJFD8EBT:~# ./builtin_shell
===== BUILT-IN COMMAND DISPATCH TABLE =====
Type 'help' to see commands.
Type 'exit' to quit.

builtin-shell> help

===== BUILT-IN COMMANDS =====
cd <directory> - Change directory
pwd           - Show current directory
echo <text>    - Display text
export N=V    - Set environment variable
help         - Display commands
exit         - Exit shell

builtin-shell> pwd
Current directory: /root

builtin-shell> echo hello
hello

builtin-shell> cd /tmp

builtin-shell> pwd
Current directory: /tmp

builtin-shell> cd /root

builtin-shell> pwd
Current directory: /root

builtin-shell> export COURSE=OSSP
Exported COURSE=OSSP

builtin-shell> hello
Invalid command: hello

builtin-shell> exit
Exiting built-in shell...
root@LAPTOP-RJFD8EBT:~# |
```

TASK-3:

To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <limits.h>


#define INPUT_SIZE 256


/* Display current working directory */
void display_current_directory(void)
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("getcwd");
        return;
    }

    printf("Current directory: %s\n", cwd);
}


/* Save current directory as state */
```

```
int save_state(char *state, size_t size)
```

```
{  
    if (getcwd(state, size) == NULL)  
    {  
        perror("getcwd");  
        return 0;  
    }  
  
    return 1;  
}
```

```
/* Cleanup resources */
```

```
void cleanup(char *state)
```

```
{  
    if (state != NULL)  
    {  
        free(state);  
        printf("Resources cleaned up successfully.\n");  
    }  
}
```

```
int main(void)
```

```
{  
    char input[INPUT_SIZE];  
    char *saved_state;
```

```
printf("==== CURRENT DIRECTORY AND EXIT HANDLING =====\n");
```

```
/* Allocate memory for saved state */
```

```
saved_state = malloc(PATH_MAX);
```

```
if (saved_state == NULL)
```

```
{
```

```
    perror("malloc");
```

```
    return 1;
```

```
}
```

```
/* Save initial directory */
```

```
if (!save_state(saved_state, PATH_MAX))
```

```
{
```

```
    cleanup(saved_state);
```

```
    return 1;
```

```
}
```

```
printf("Initial directory state saved.\n");
```

```
printf("Saved path: %s\n", saved_state);
```

```
printf("\nAvailable commands:\n");
```

```
printf("  pwd - Display current directory\n");
```

```
printf("  cd PATH - Change directory\n");
```

```
printf("  state - Display saved directory state\n");
```

```
printf("  exit - Exit and cleanup resources\n");
```

```
while (1)
{
    printf("\npwd-shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("\nInput ended. Exiting...\n");
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Ignore empty input */
    if (strlen(input) == 0)
    {
        continue;
    }

    /* pwd command */
    if (strcmp(input, "pwd") == 0)
    {
        display_current_directory();
    }
}
```

```
/* state command */  
else if (strcmp(input, "state") == 0)  
{  
    printf("Saved directory state: %s\n", saved_state);  
}
```

```
/* exit command */  
else if (strcmp(input, "exit") == 0)  
{  
    printf("Exit request received.\n");  
    printf("Performing cleanup...\n");  
    break;  
}
```

```
/* cd command */  
else if (strncmp(input, "cd ", 3) == 0)  
{  
    char *path = input + 3;  
  
    if (strlen(path) == 0)  
    {  
        printf("Error: directory path is missing.\n");  
        continue;  
    }
```

```
    if (chdir(path) != 0)
```

```
{
    perror("cd");
    continue;
}

printf("Directory changed successfully.\n");

/* Update saved state */
if (save_state(saved_state, PATH_MAX))
{
    printf("State updated: %s\n", saved_state);
}
}

/* Invalid command */
else
{
    printf("Invalid command: %s\n", input);
}
}

/* Cleanup before exiting */
cleanup(saved_state);

printf("Program terminated successfully.\n");
```

```
    return 0;
}
```

```
root@LAPTOP-RJFD8EBT:~# nano pwd_exit.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g pwd_exit.c -o pwd_exit
root@LAPTOP-RJFD8EBT:~# ./pwd_exit
===== CURRENT DIRECTORY AND EXIT HANDLING =====
Initial directory state saved.
Saved path: /root

Available commands:
  pwd - Display current directory
  cd PATH - Change directory
  state - Display saved directory state
  exit - Exit and cleanup resources

pwd-shell> pwd
Current directory: /root

pwd-shell> cd /tmp
Directory changed successfully.
State updated: /tmp

pwd-shell> pwd
Current directory: /tmp

pwd-shell> state
Saved directory state: /tmp

pwd-shell> cd /does_not_exist
cd: No such file or directory

pwd-shell> hello
Invalid command: hello

pwd-shell> exit
Exit request received.
Performing cleanup...
Resources cleaned up successfully.
Program terminated successfully.
root@LAPTOP-RJFD8EBT:~# |
```

TASK-4:

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

#include <unistd.h>

#include <sys/wait.h>

#define INPUT_SIZE 256

int valid_name(const char *name)
{
```

```

int i;

if (name == NULL || name[0] == '\0')
    return 0;

if (!(isalpha((unsigned char)name[0]) || name[0] == '_'))
    return 0;

for (i = 1; name[i] != '\0'; i++)
{
    if (!(isalnum((unsigned char)name[i]) || name[i] == '_'))
        return 0;
}

return 1;
}

int process_export(char *command)
{
    char *assignment;
    char *name;
    char *value;
    char *equal;

    /* Check export syntax */
    if (strncmp(command, "export", 6) != 0)
    {
        return 0;
    }

    if (command[6] != '\0' && command[6] != ' ')

```

```
{  
    printf("Invalid command syntax.\n");  
    return 1;  
}
```

```
assignment = command + 6;  
while (*assignment == ' ')  
    assignment++;
```

```
if (*assignment == '\0')  
{  
    printf("Usage: export NAME=VALUE\n");  
    return 1;  
}
```

```
equal = strchr(assignment, '=');
```

```
if (equal == NULL)  
{  
    printf("Invalid syntax. Use: export NAME=VALUE\n");  
    return 1;  
}
```

```
name = assignment;  
value = equal + 1;
```

```
/* Validate variable name */
```

```
if (!valid_name(name))
{
    printf("Invalid variable name: %s\n", name);
    return 1;
}
if (getenv(name) != NULL)
{
    printf("Variable '%s' already exists.\n", name);
    printf("Updating its value...\n");
}
else
{
    printf("Creating new variable '%s'.\n", name);
}
if (setenv(name, value, 1) != 0)
{
    perror("setenv");
    return 1;
}

printf("Export successful: %s=%s\n", name, value);

return 1;
}

void show_variable(const char *name)
{

```

```

char *value = getenv(name);

if (value != NULL)
    printf("%s=%s\n", name, value);
else
    printf("%s is not defined.\n", name);
}

/* Test whether child process receives environment variable */
void test_child_process(const char *name)
{
    pid_t pid;
    int status;

    printf("\nCreating child process to test environment...\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {

```

```

    printf("Child PID: %d\n", getpid());
    printf("Child received: ");

    show_variable(name);

    exit(0);
}

waitpid(pid, &status, 0);

if (WIFEXITED(status))
{
    printf("Child process completed successfully.\n");
}
}

int main(void)
{
    char input[INPUT_SIZE];
    char *value;

    printf("==== EXPORT ENVIRONMENT VARIABLE DEMONSTRATION\n");

    printf("\nCommands:\n");
    printf(" export NAME=VALUE - Create/update variable\n");
    printf(" show NAME      - Display variable\n");

```

```
printf(" child NAME      - Test variable in child process\n");
```

```
printf(" exit          - Exit program\n");
```

```
while (1)
```

```
{
```

```
    printf("\nexport-shell> ");
```

```
    fflush(stdout);
```

```
    if (fgets(input, sizeof(input), stdin) == NULL)
```

```
    {
```

```
        break;
```

```
    }
```

```
    input[strcspn(input, "\n")] = '\0';
```

```
    /* Ignore empty input */
```

```
    if (strlen(input) == 0)
```

```
        continue;
```

```
    /* Exit */
```

```
    if (strcmp(input, "exit") == 0)
```

```
    {
```

```
        printf("Exiting export shell...\n");
```

```
        break;
```

```
    }
```

```
    if (strncmp(input, "show ", 5) == 0)
```

```

{
    char *name = input + 5;

    if (valid_name(name))
        show_variable(name);
    else
        printf("Invalid variable name: %s\n", name);

    continue;
}

if (strncmp(input, "child ", 6) == 0)
{
    char *name = input + 6;

    if (valid_name(name))
        test_child_process(name);
    else
        printf("Invalid variable name: %s\n", name);

    continue;
}

/* Export command */
if (strncmp(input, "export", 6) == 0)
{
    process_export(input);
}

```

```

        continue;
    }

    printf("Invalid command: %s\n", input);
}

value = getenv("COURSE");

if (value != NULL)
{
    printf("\nFinal COURSE value: %s\n", value);
}

printf("Program terminated successfully.\n");

return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano export_demo.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g export_demo.c -o export_demo
root@LAPTOP-RJFD8EBT:~# ./export_demo
===== EXPORT ENVIRONMENT VARIABLE DEMONSTRATION =====

Commands:
  export NAME=VALUE  - Create/update variable
  show NAME          - Display variable
  child NAME         - Test variable in child process
  exit              - Exit program

export-shell> export COURSE=OSSP
Creating new variable 'COURSE'.
Export successful: COURSE=OSSP

export-shell> show COURSE
COURSE=OSSP

export-shell> export COURSE=OPERATING_SYSTEMS
Variable 'COURSE' already exists.
Updating its value...
Export successful: COURSE=OPERATING_SYSTEMS

export-shell> show COURSE
COURSE=OPERATING_SYSTEMS

export-shell> export STUDENT=Rajini
Creating new variable 'STUDENT'.
Export successful: STUDENT=Rajini

export-shell> show STUDENT
STUDENT=Rajini

export-shell> export 123NAME=value
Invalid variable name: 123NAME

export-shell> export MY-VAR=value
Invalid variable name: MY-VAR

export-shell> export COURSE
Invalid syntax. Use: export NAME=VALUE

export-shell> export PROJECT=OSSP
Creating new variable 'PROJECT'.
Export successful: PROJECT=OSSP

export-shell> show PROJECT
PROJECT=OSSP

```

```

Export successful: COURSE=OPERATING_SYSTEMS

export-shell> show COURSE
COURSE=OPERATING_SYSTEMS

export-shell> export STUDENT=Rajini
Creating new variable 'STUDENT'.
Export successful: STUDENT=Rajini

export-shell> show STUDENT
STUDENT=Rajini

export-shell> export 123NAME=value
Invalid variable name: 123NAME

export-shell> export MY-VAR=value
Invalid variable name: MY-VAR

export-shell> export COURSE
Invalid syntax. Use: export NAME=VALUE

export-shell> export PROJECT=OSSP
Creating new variable 'PROJECT'.
Export successful: PROJECT=OSSP

export-shell> show PROJECT
PROJECT=OSSP

export-shell> child PROJECT

Creating child process to test environment...
Child PID: 3012
Child received: PROJECT=OSSP
Child process completed successfully.

export-shell> exit
Exiting export shell...

Final COURSE value: OPERATING_SYSTEMS
Program terminated successfully.
root@LAPTOP-RJFD8EBT:~# |

```